

Programming in C and C++

The 80/20 Revision Guide

Part IB — Cambridge CST

How this guide is organised

The Cambridge Part IB C/C++ paper (Paper 4) sets two questions per year. Looking at **eight years of past papers (2018–2025, 16 questions)**, the topic frequencies are:

Topic	Appearances	Share
Memory allocation & management (<code>malloc/free</code> , custom arenas)	9	56%
Pointers & dereferencing	6	38%
Linked lists & self-referential structs in C	4	25%
Low-level / embedded (<code>volatile</code> , bit fields, hardware)	3	19%
Templates and generics (C++)	3	19%
Data types & bit manipulation	3	19%
Inheritance & polymorphism (C++)	2	13%
Strings	2	13%
Type casting / safety in C++	1	6%
Operator overloading	1	6%
Function definitions: C vs C++	1	6%

80/20 verdict. Memory management + pointers + linked structures account for roughly **75% of marks**. If you only revise three things, revise those. Templates, inheritance, and bit-level C are the next tier and reliably feature once every other year. Everything else (preprocessor, enums, history, etc.) is supporting material — worth a paragraph each.

This document is sectioned in that order: **must-know first, then important, then context**.

1 The Big Three (75% of marks)

1.1 Memory: `malloc`, `free`, and lifetimes

Exam-critical

The single most likely thing to appear is a question that gives you a data structure (often a tree, DAG, or linked list) and asks: (a) write code to construct it; (b) write code to free it; (c) explain what goes wrong if pointers alias / if there's a cycle; (d) propose a better scheme (arena, reference counting, GC).

The C dynamic allocation API lives in `<stdlib.h>`:

```
void *malloc(size_t size);           // uninitialised bytes
void *calloc(size_t n, size_t size); // zeroed
void *realloc(void *p, size_t size); // resize
void free(void *p);                  // exactly once per allocation
```

The contract is brutal but simple:

- Every successful `malloc` must be matched by exactly one `free`, **on every execution path**.

- After `free(p)`, `p` must not be used (use-after-free) and `free(p)` must not be called again (double-free).
- Failure to free $\Rightarrow$ memory leak. Free too early $\Rightarrow$ dangling pointer. Free twice $\Rightarrow$ heap corruption.

Gotcha

A classic exam trap: `free` is conditional on a branch, so one path through the program leaks.

```
int *pi = malloc(sizeof(int));
scanf("%d", pi);
if (*pi % 2) {
    printf("Odd!\n");
    free(pi);    // WRONG: only freed on odd input
}
// Fix: move free(pi) outside the if.
```

Trees: easy. DAGs: hard.

For a binary tree where every node has at most one parent, recursive free is fine:

```
void tree_free(Tree *t) {
    if (t != NULL) {
        tree_free(t->left);
        tree_free(t->right);
        free(t);
    }
}
```

But the moment two pointers **alias** the same node (a DAG), this recursive free will free that node twice. **This is the canonical lecture example** (2024-4-6, 2020-4-1 ask variants).

Three standard solutions — know all of them:

(1) Visited flag / explicit graph traversal

Add a `visited` field. Walk the graph, collect unique nodes into a list, free each one once.

```
TreeList *getNodes(Tree *t, TreeList *acc) {
    if (t == NULL || t->visited) return acc;
    t->visited = true;
    acc = cons(t, acc);
    acc = getNodes(t->right, acc);
    acc = getNodes(t->left, acc);
    return acc;
}
```

(2) Arena allocation

Allocate a big block up front; hand out nodes from it; free the whole arena in one go. **Sidesteps aliasing entirely** because we never free individual nodes.

```
typedef struct arena {
    int size, current;
    Tree *elts;
} *arena_t;

Tree *node(int v, Tree *l, Tree *r, arena_t a) {
    if (a->current >= a->size) return NULL;
    Tree *t = &a->elts[a->current++];
    t->value = v; t->left = l; t->right = r;
    return t;
}
```

```

}

void free_arena(arena_t a) { free(a->elts); free(a); }

```

Trade-off: can't free individual nodes; arena holds memory until the very end (bigger working set). Best when nodes have a common lifetime.

(3) Reference counting

Each object carries an integer count of incoming pointers. Increment on copy, decrement on drop, free when it hits zero. Recursively decrements counts of children.

```

typedef struct rc_node {
    int refcount;
    int value;
    struct rc_node *left, *right;
} RCNode;

void rc_decref(RCNode *n) {
    if (n == NULL) return;
    if (--n->refcount == 0) {
        rc_decref(n->left);
        rc_decref(n->right);
        free(n);
    }
}

```

Achilles' heel: reference cycles leak. Use weak references or a cycle collector if cycles are possible.

1.2 Pointers

Exam-critical

A pointer is a **variable whose value is an address**. The pointer itself has a size (typically 8 bytes on 64-bit machines) regardless of what it points to. **Almost every C exam question hinges on tracking what each pointer aims at, step by step.**

Declaration, dereference, address-of

```

int x = 5;
int *p = &x;    // p holds the address of x
int **pp = &p;   // pp holds the address of p
*p = 10;         // x is now 10
**pp = 20;       // x is now 20

```

The asterisk binds to the *name*, not the type — so `int *a, b;` declares one `int*` and one `int`.

Pointers and arrays

An array name decays to a pointer to its first element in almost every expression. Therefore:

```

int arr[10];
int *p = arr;    // equivalent to &arr[0]
arr[i] == *(arr + i) == *(p + i) == p[i];

```

But: `arr` is not a variable, so `arr = p` and `arr++` are **errors**, while `p = arr` and `p++` are fine.

Pointer arithmetic

`p + n` advances `p` by `n * sizeof(*p)` bytes, not `n` bytes. This means pointer arithmetic is *type-aware*: incrementing an `int*` by 1 on a 32-bit-int machine actually adds 4 to the address. Out-of-bounds pointer arithmetic is undefined.

Pass-by-value, pointer trickery

All function arguments in C are passed by value. To mutate the caller's variable, pass a pointer to it:

```
void swap_bad(int x, int y) { int t=x; x=y; y=t; }      // no effect
void swap_ok (int *x, int *y) { int t=*x; *x=*y; *y=t; } // works
// caller: swap_ok(&a, &b);
```

Function pointers

```
int  cmp(int a, int b) { return a > b ? 1 : 0; }
void sort(int a[], int n, int (*lt)(int,int));
sort(arr, n, cmp);      // pass function as argument
```

The compiler accepts both `(*lt)(x,y)` and `lt(x,y)`. Function pointers underpin polymorphic C (qsort, callbacks, vtables).

void * — C's escape hatch

`void *` holds an address with no type. `malloc` returns one. You need a cast (or implicit conversion in C, not C++) to use it:

```
int *p = malloc(n * sizeof(int)); // C: implicit
int *p = (int *)malloc(n * sizeof(int)); // required in C++
```

Gotcha

Returning a pointer to a local variable is undefined — the stack frame disappears on return:

```
char *unary(unsigned short s) {
    char local[s+1];          // on the stack!
    /* fill local ... */
    return local;              // DOOM: dangling pointer
}
```

Either `malloc` the buffer, or have the caller pass one in.

1.3 Linked structures in C

Exam-critical

Memory questions almost always set themselves on top of a linked list, tree, or graph. Be fluent in defining a self-referential struct, walking it, inserting into it, and freeing it.

```
struct node {
    int value;
    struct node *next;      // self-referential via pointer
};
typedef struct node Node;

Node *push(Node *head, int v) {
```

```

Node *n = malloc(sizeof(Node));
n->value = v;
n->next = head;
return n;           // new head
}

void list_free(Node *head) {
    while (head != NULL) {
        Node *nxt = head->next;
        free(head);
        head = nxt;
    }
}

```

For trees, use two child pointers (**left**, **right**). The standard idioms are:

- **Recursive insert:** compare value, recurse left or right.
- **Recursive print:** in-/pre-/post-order traversal.
- **Recursive free:** post-order (free children before parent — otherwise you lose the pointers).

Member access

`n.value` for a struct, `n->value` for a pointer to a struct. They mean: `n->value == (*n).value`. The arrow is just sugar.

2 The Second Tier

2.1 C++ Templates and Generics

Worth knowing

Appears in 2020-4-2, 2024-4-6, 2025-4-6 — roughly every other year. Usually paired with operator overloading or inheritance to make a small generic container or numeric class.

A template is a recipe the compiler instantiates per type:

```

template <typename T>
class Stack {
    T *data;
    int top, capacity;
public:
    Stack(int n) : top(0), capacity(n) { data = new T[n]; }
    ~Stack() { delete[] data; }
    void push(const T& x) { data[top++] = x; }
    T pop() { return data[--top]; }
    bool empty() const { return top == 0; }
};

Stack<int> si(100);           // instantiates Stack<int>
Stack<double> sd(100);       // a separate instantiation

```

Key things to know in the exam:

- Templates are *compile-time* polymorphism — the compiler generates separate code per type. No runtime overhead, but code bloat.
- Contrast with C: in C you'd use `void *` (no type safety) or macros (no type checking).

- Function templates: `template <typename T> T max(T a, T b) { return a < b ? b : a; }`
- Templates require the type to support the operations used — e.g. a `Stack<T>` needs `T` to be assignable.

2.2 Classes, RAII, destructors

Worth knowing

The headline difference from C: C++ has constructors and destructors that run automatically. **RAII** (Resource Acquisition Is Initialisation) is the entire C++ memory story — objects acquire resources in the constructor and release them in the destructor, tied to scope.

```
class Buffer {
    char *data;
    int size;
public:
    Buffer(int n) : size(n) { data = new char[n]; }
    ~Buffer() { delete[] data; } // auto-runs on scope exit
    // Rule of three: also need copy ctor & copy assignment
    Buffer(const Buffer& o);
    Buffer& operator=(const Buffer& o);
};

void f() {
    Buffer b(1024); // constructor runs
    // ... use b ...
} // destructor runs here, even on exception
```

The rule of three: if you need to write a custom destructor, you probably also need a custom copy constructor and a custom copy assignment operator. Otherwise the compiler's default shallow-copy will double-free your heap data.

References vs pointers

```
int x = 5;
int *p = &x; // pointer: explicit address
int &r = x; // reference: alias for x
```

A reference must be bound at creation, can never be null, and can never be re-bound. Use references for: function parameters that mustn't be null, return-by-reference, operator overloading. Use pointers when you need null, re-binding, or pointer arithmetic.

2.3 Inheritance, virtual functions, polymorphism

Worth knowing

Inheritance shows up in 2020-4-2, 2024-4-6. Questions tend to ask: explain what `virtual` does, what the vtable looks like, why slicing happens, what `override` buys you.

```
class Shape {
public:
    virtual double area() const = 0; // pure virtual
    virtual ~Shape() = default; // virtual dtor required for polymorphic base
};

class Circle : public Shape {
    double r;
```

```
public:
    Circle(double r) : r(r) {}
    double area() const override { return 3.14159 * r * r; }
};
```

What you must remember:

- **virtual** means: dispatch is decided at runtime via a **vtable** (a per-class array of function pointers; each object stores a pointer to its class's vtable).
- **Without virtual**, calls resolve statically based on the declared type — so `Shape s = c; s.area();` (object slicing!) gives the *Shape* version, not the *Circle* one.
- **Pure virtual** (= 0) makes the class abstract — you can't instantiate it directly, only derive from it.
- **Polymorphism only works via pointers or references**. A by-value `Shape` variable is sliced down to the base class.
- Always declare base-class destructors **virtual** — otherwise `delete base_ptr` won't call the derived destructor.

2.4 Bit fields, volatile, low-level work

Worth knowing

The “low-level / embedded” flavour appears in 2023-4-6, 2024-4-5, 2025-4-5. Often: “you're driving a hardware register, write the struct.”

Bit fields pack named sub-fields of a word:

```
struct status_reg {
    unsigned int ready    : 1;    // 1 bit
    unsigned int error    : 1;
    unsigned int reserved : 6;
    unsigned int channel  : 8;    // 8 bits
};
```

Caveats: bit-field layout (endianness, padding) is implementation-defined. You can't take the address of a bit field (no `&`).

volatile tells the compiler “this value may change outside the program's control” — e.g. a memory-mapped device register, a flag set by a signal handler, or a variable read by another thread. The compiler must not optimise away reads or reorder accesses across it.

```
volatile uint32_t *uart_status = (uint32_t *)0x4000B000;
while ((*uart_status & READY_BIT) == 0) { }    // must re-read each iteration
```

const placement is subtle — read right-to-left:

```
const int *p;        // p points to const int (can't write *p, can re-aim p)
int *const p;        // const pointer to int (can write *p, can't re-aim)
const int *const p;  // const pointer to const int (can't do either)
```

2.5 Data types and bit manipulation

Worth knowing

Bit twiddling appears in 2018-4-1, 2021-4-1, 2023-4-5. Questions: extract a field from a packed int, count set bits, swap nibbles, convert endianness.

The basic types: `char` (≥ 8 bits), `short`, `int` (≥ 16 , usually word-sized), `long`, `long long`, `float`, `double`. Exact widths are architecture-defined — use `<stdint.h>` types (`int32_t`, `uint64_t`) when size matters.

Bitwise operators: `&` AND, `|` OR, `^` XOR, `~` NOT, `<<` left shift, `>>` right shift.

Standard idioms:

```
x & (1 << n)           // test bit n
x | (1 << n)           // set bit n
x & ~(1 << n)          // clear bit n
x ^ (1 << n)           // flip bit n
(x >> n) & MASK         // extract a field
(x >> 24) | ((x & 0xFF0000) >> 8) | ((x & 0xFF00) << 8) | (x << 24)
                        // byte-swap a 32-bit value (endianness flip)
```

Signed vs unsigned shifts: right-shifting a signed negative number is implementation-defined (usually arithmetic shift, sign-extending). Right-shifting an unsigned is always logical (zero-fill). Prefer unsigned types for bit work.

Integer overflow: signed overflow is **undefined behaviour**; unsigned overflow wraps modulo 2^n . Don't assume `i + 1 > i` for signed `i` — a compiler can (and does) delete checks that rely on it.

3 Cache, performance, undefined behaviour, tooling

3.1 Memory hierarchy and data layout

Code is fast when the cache is happy. Two big levers:

Array-of-structs (AoS) vs struct-of-arrays (SoA).

```
struct Particle { float x, y, z; float vx, vy, vz; } p[N]; // AoS
struct { float x[N], y[N], z[N], vx[N], vy[N], vz[N]; } P; // SoA
```

If you stream over only the positions, SoA wins: contiguous, no wasted cache bytes for velocities. If you touch all six per particle in the same loop, AoS wins.

Loop blocking (tiling). For a nested loop that touches a 2D array, process in $B \times B$ tiles that fit in cache, rather than full rows:

```
for (int ii = 0; ii < N; ii += B)
    for (int jj = 0; jj < N; jj += B)
        for (int i = ii; i < ii+B; i++)
            for (int j = jj; j < jj+B; j++)
                A[i][j] = ...;
```

Intrusive lists. Embed the linked-list pointers *inside* the payload struct, rather than allocating a separate list cell holding a pointer to the payload. One allocation per object instead of two; better locality.

3.2 Undefined behaviour and tooling

C is unsafe by design. **Undefined behaviour** (UB) means the compiler may do *anything* — crash, return garbage, silently delete code. Sources include: signed overflow, dereferencing NULL, out-of-bounds access, use-after-free, double-free, reading uninitialised memory, breaking strict aliasing.

Four tools you should be able to name:

Tool	Catches	Cost
ASan	out-of-bounds, use-after-free, double-free, leaks	2× slowdown, recompile
MSan	reads of uninitialised memory	2–3×, recompile, clang only
UBSan	signed overflow, null deref, bad shifts	~20%, recompile
Valgrind	memory errors via binary instrumentation	10–50×, no recompile

Use them with the relevant `-fsanitize=address`, `-fsanitize=memory`, `-fsanitize=undefined` flags, or invoke `valgrind ./a.out`.

3.3 Garbage collection in C

A type-specific GC for, e.g., trees: track all live root pointers; periodically walk reachable objects (mark phase); free everything not marked (sweep phase). Conservative GCs (e.g. Boehm) treat anything word-aligned in memory that looks like a pointer as one. Useful but pessimistic.

4 Supporting cast (rarely asked but expected background)

4.1 Preprocessor

```
#include <stdio.h>           // < > searches system paths
#include "myheader.h"        // " " searches local first
#define PI 3.14159
#define SQUARE(x) ((x)*(x)) // ALWAYS parenthesise macro args!
#ifdef DEBUG
    printf("debug...\n");
#endif
```

Preprocessing is pure text substitution *before* the compiler sees anything — so type errors in macros show up at the use site, not the definition.

Header guards (or `#pragma once`) prevent double-inclusion:

```
#ifndef MYHEADER_H
#define MYHEADER_H
/* declarations */
#endif
```

4.2 Compilation model

Each `.c` file compiles independently to an `.o` object file. The linker resolves cross-file references. Headers carry declarations; `.c` files carry definitions. `extern` says “declared here, defined elsewhere”; `static` at file scope says “private to this file — don’t export.”

4.3 Storage classes

- `auto` (default): local variable, lives on the stack, vanishes on return.

- **static local**: persists across calls, single instance even with recursion.
- **static file-scope global**: not exported by the linker.
- **extern**: declared here, defined elsewhere.
- **register**: hint to the compiler — mostly useless on modern hardware.

4.4 The classic differences C → C++

C	C++
struct only groups data	struct can have methods (default public); class same but default private
malloc / free	new / delete ; new[] / delete[]
No function overloading	Yes; name-mangling makes it work at link time
No default arguments	Yes: <code>void f(int x, int y = 0);</code>
No references	Yes (<code>int &r = x;</code>)
No templates	Yes
No classes, no inheritance, no virtual	Yes, yes, yes
<code>void *</code> implicit conversion to other pointers	Requires explicit cast
<code>int f()</code> means “f takes anything”	<code>int f()</code> means “f takes nothing”

5 The one-page exam checklist

Exam-critical

If you have one hour to revise, do these in order:

1. Be able to define, traverse, and free a linked list and a binary tree from blank paper.
2. Be able to explain — with a diagram — what goes wrong when two pointers alias the same heap node and you free recursively.
3. Be able to write an **arena allocator** (struct with size/current/elts; `node()` bumps a pointer; `free_arena()` frees once).
4. Be able to write **reference counting** primitives (incref, decref, free-when-zero) and explain the cycle-leak problem.
5. Be able to read `const int * const *p` and similar.
6. Be able to write a templated `Stack<T>` class with a constructor, destructor, push, pop.
7. Know what **virtual** does, why the base destructor must be virtual, and what slicing is.
8. Know all four bit-twiddle idioms (set / clear / test / flip / extract).
9. Be able to name and describe ASan, MSan, UBSan, Valgrind and what each catches.
10. Be able to give two examples of undefined behaviour and explain why the compiler is allowed to do anything.

Good luck.